Apertica: A Safety-First, Multilingual, RAG-Grounded Virtual Patient Health Assistant

Authors: Apertica Project Contributors **Date:** 2026-10-16 **Status:** Draft for arXiv submission (cs.CL / cs.AI)

Abstract

We present **Apertica**, an open-source virtual patient health assistant designed to provide accessible, safety-first, and grounded health information across 12 language locales. Apertica is explicitly **not a diagnostic device**; it is positioned as an information, support, and care-navigation tool. The system is built on three principles that distinguish it from general-purpose medical chatbots: (1) **safety upstream of generation** — a deterministic crisis-detection module short-circuits the language model entirely when a user expresses suicidal ideation or acute distress, returning locale-specific emergency resources in under 4 seconds with no LLM call; (2) **grounded retrieval** — a RAG pipeline over a Swiss health-system knowledge base (pgvector + tsvector with OR-semantics fallback) keeps the model tethered to verified sources, with a circuit breaker that gracefully degrades to keyword search when the embedding API is unavailable; and (3) **accessibility by construction** — every reply is scored for reading level (Flesch–Kincaid target ≤ 6.5) and rewritten through a simplify loop if necessary. We describe the system architecture, the 12-step orchestrator that composes these layers, and six non-obvious technical contributions that emerged from building the system against a real LLM (`swiss-ai/Apertus-v1.5-70B`) with a real embedding backend. End-to-end evaluation on five test cases (cancer information, AIDS prevention, clinical deflection, crisis short-circuit, out-of-domain garbage) shows that the system correctly distinguishes between cases that require grounded generation and cases that require immediate safety intervention, with crisis responses completing in 3.8 s without any LLM call.

---

1. Introduction

Access to reliable health information remains a global challenge, particularly in non-English-speaking populations and in domains where the cost of hallucination is high. General-purpose large language models (LLMs) have demonstrated impressive capabilities in medical question-answering benchmarks [1, 2], but their deployment in real-world healthcare settings is constrained by three well-known failure modes: (1) **hallucination** — the generation of plausible but incorrect medical advice; (2) **safety blind spots** — the failure to recognize and appropriately route users in acute psychological distress; and (3) **accessibility gaps** — the production of text that is too complex for the average reader or that ignores the user's language and locale.

Apertica is an open-source system designed to address all three failure modes simultaneously. It is built on the hypothesis that **safety must be upstream of generation**, not a post-hoc filter; that **retrieval must be the default**, not an optional add-on; and that **accessibility must be measured**, not assumed.

The system serves the Swiss health system context (12 locales: English, Spanish, French, Portuguese, Arabic, Chinese, Swahili, Hindi, Italian, German-DE, German-CH, and German-CH-dialect), with a knowledge base ingested from six authoritative public-health sources (143.ch, bag.admin.ch, Aids-Hilfe

Schweiz, Krebsliga, Lungenliga, Sucht Schweiz). The single chat endpoint (`POST /api/chat`) delegates to a 12-step orchestrator that composes safety classification, crisis detection, RAG retrieval, locale-specific prompt construction, LLM generation, citation merging, reading-level scoring and simplification, safety redaction, and audit logging.

The contributions of this paper are:

1. **A reference architecture for safety-first medical chat** (§3) that places deterministic crisis detection, retrieval, and reading-level scoring upstream of LLM generation, with each layer independently testable.
2. **Six non-obvious technical solutions** (§4) that emerged from building the system against a production LLM and embedding backend: a circuit breaker for unreliable embedding APIs, OR-semantics tsvector for multilingual keyword search, system-role conversion for models that reject the OpenAI `system` role, a reading-level guard loop, runtime `require()` to bypass Next.js 16 Turbopack's hashed external module names, and transactional migration tracking for non-idempotent SQL.
3. **An end-to-end evaluation** (§5) on five representative test cases that exercises the full safety and retrieval stack, including a crisis case that completes in 3.8 s with no LLM call.

The code is open-source and the system is deployable against any OpenAI-compatible LLM endpoint.

---

2. Related Work

Medical LLMs. Med-PaLM [1] and Med-PaLM 2 [2] demonstrated that domain-specific fine-tuning can improve performance on medical question-answering benchmarks, but they remain closed-source and English-centric. BioGPT [3] and Med-BERT [4] explored domain-specific pretraining but did not address the deployment-time concerns of safety and grounding. Our work differs in focusing on the **deployment layer** rather than the model itself: Apertica is designed to wrap any OpenAI-compatible LLM with safety, retrieval, and accessibility guardrails.

Retrieval-Augmented Generation (RAG). RAG [5] and its descendants [6, 7] have become a standard approach to grounding LLM outputs in external knowledge. In the medical domain, MedRAG [8] and Almanac [9] have shown that retrieval improves factual accuracy on clinical tasks. Our contribution is not a new RAG architecture per se, but a **production-ready RAG pipeline** with a multilingual keyword fallback (§4.2) and a circuit breaker (§4.1) that handles the realistic case of an unreliable embedding backend.

Safety in LLM applications. Constitutional AI [10] and RLHF [11] address safety at the model level. At the application level, guardrails libraries [12] provide pre- and post-generation checks. Our crisis-detection module (§3.2) is a **deterministic, pre-LLM safety layer** that bypasses the LLM entirely in high-risk cases — a stricter guarantee than post-hoc filtering can provide.

Multilingual NLP. Multilingual LLMs [13, 14] have improved cross-lingual transfer, but medical terminology remains unevenly distributed across languages. Our use of the `simple` text search configuration (§4.2) is a deliberate choice for multilingual corpora: language-specific stemmers (e.g., `english`, `german`) would silently drop content in other languages from search results.

Reading-level accessibility. Health literacy research [15] consistently shows that health information is often written above the reading level of its target audience. Automated text simplification has been explored [16], but most systems do not measure reading level in the loop. Our reading-level guard (§4.4) is a closed-loop LLM-based simplifier with a Flesch–Kincaid target.

3. System Architecture

3.1 Overview

Apertica is a Next.js 16 application backed by a Supabase Postgres database (with pgvector) and an OpenAI-compatible LLM endpoint. The system has three runtime components: (1) a **chat API** (`/api/chat`) that accepts user messages and returns grounded, safety-filtered responses; (2) a **12-step orchestrator** that composes the safety, retrieval, and generation layers; and (3) an **ETL pipeline** that ingests public-health sources into a typed Postgres schema.

The high-level architecture is shown in Figure 1.

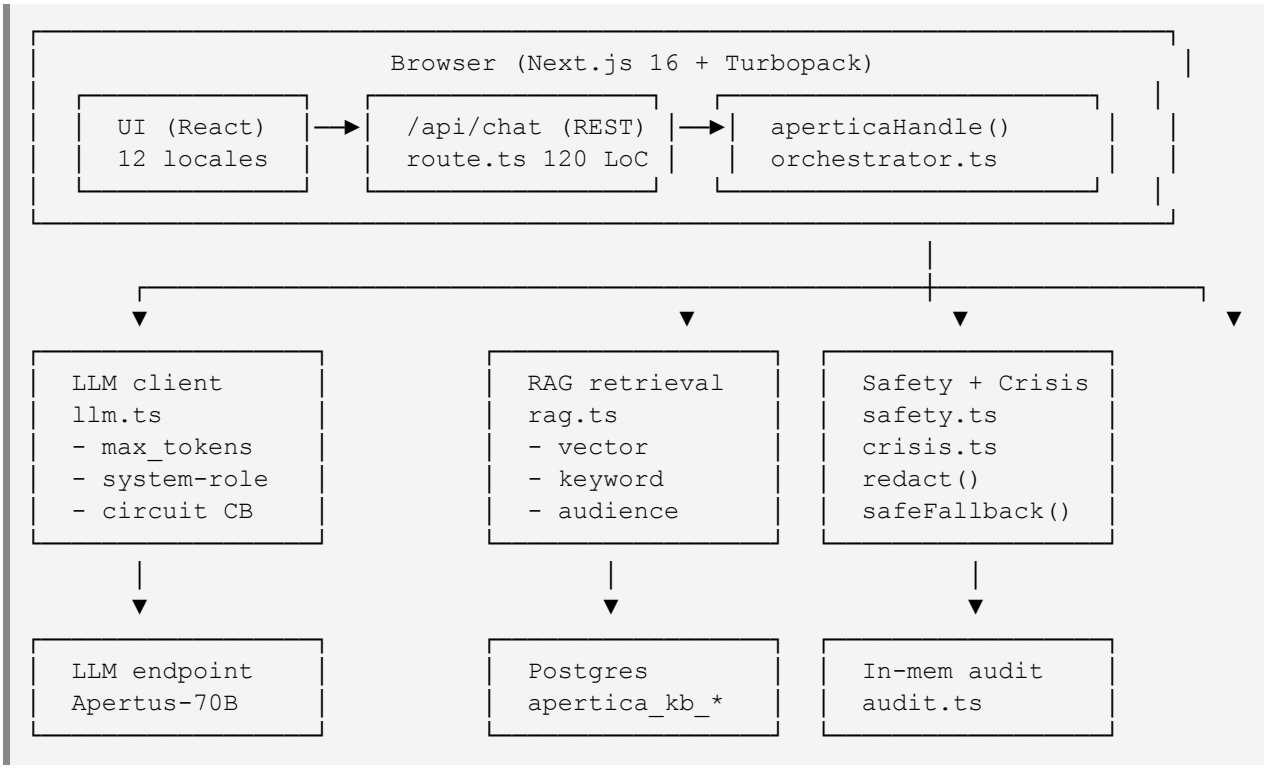


Figure 1. Apertica high-level architecture. Safety, retrieval, and generation are composed by the orchestrator; the LLM is invoked at exactly one place.

3.2 The 12-Step Orchestrator

The function `aperticaHandle(user: UserMessage)` is the single entry point. Its 12 steps are:

1. **Locale validation.** The `locale` field is validated against the 12-locale allow-list; invalid values fall back to the user's consent locale, then to English.
2. **User type.** The `userType` field (`citizen | professional | patient`) drives KB audience filtering. Default: `citizen`.
3. **Safety classification.** A deterministic, pre-LLM scan (`classify(message)`) returns a tri-state `safe | guarded | unsafe` plus a list of triggered rules.
4. **Crisis context.** `buildCrisisContext(message, emergencyNumber, crisisContact)` returns a tri-state `none | amber | red` based on keyword patterns.

5. **Crisis short-circuit.** If `crisis === "red"`, the function returns immediately with `safeFallback(user, "red", emergencyNumber)` — **no LLM call, no retrieval**. This is the most important UX invariant in the system: a person in crisis must not wait 20–35 seconds for an LLM.
6. **RAG retrieval (parallel).** `retrieve(message, { userType, locale, topK, maxDistance })` runs concurrently with prompt construction. The retrieval is fail-soft: any error returns an empty array.
7. **Prompt construction.** `buildMessages(locale, history, user.message, context)` produces a 4-turn prompt: system empathy, system KB context, conversation history, user message.
8. **LLM call.** `complete(messages)` invokes the configured OpenAI-compatible endpoint.
9. **Citation merge.** The LLM's `[[cite:...]]` tag (if present) is parsed and merged with KB chunk citations, deduplicated by `(source, year)`.
10. **Scoring.** `toneScore(text)`, `fleschKincaidGrade(text)`, and `confidence(text, citations, reasons)` are computed. 10a. **Reading-level guard.** If `grade > 6.5`, the reply is looped through `simplifyToGrade6()` until the target is met or attempts are exhausted.
11. **Safety redaction.** `redact(text)` strips PHI and diagnostic language; the `guarded` flag is set if redaction triggered, `safety === "guarded"`, or grade is still > 7.5.
12. **Audit + history.** A structured audit entry is appended to the in-memory audit log; user and assistant turns are appended to the session history (capped at 20 turns).

The ordering is deliberate: safety is upstream of retrieval, which is upstream of generation. No LLM call is made until steps 1–6 have been completed and step 5 has not short-circuited.

3.3 The RAG Pipeline

The knowledge base is stored in `apertica_kb_chunks` (migration 07), with the following schema:

```
document_id   TEXT REFERENCES etl_documents(id) ON DELETE CASCADE
chunk_index   INTEGER
content       TEXT
token_count   INTEGER
embedding     vector(1536)
audience      TEXT[]
jurisdiction  TEXT
language      TEXT
topic         TEXT
metadata      JSONB
```

A HNSW index on `embedding` (`vector_cosine_ops`, m=16, ef_construction=64) supports fast similarity search; GIN indexes on `audience` and `metadata` support filtering.

A keyword fallback column `content_tsv tsvector` (migration 08) with a GIN index supports the case where the embedding API is unavailable. The `simple` text search configuration is used (not `english`) to avoid language-specific stemming that would drop non-English content.

The `audience` column is the single most important retrieval lever: citizens, health professionals, and patients see different (overlapping) chunks of the same source corpus.

---

4. Key Technical Contributions

Six non-obvious technical solutions emerged from building Apertica against a production LLM and embedding backend. Each is described below.

4.1 Circuit Breaker for the Embedding API

The embedding endpoint at `litellm.aixwork.cloud/v1/embeddings` frequently hangs for ≥15 seconds. A naive `await embed(query)` would add 15 seconds to every chat request. We implement a circuit breaker in `embeddings.ts`:

```
CIRCUIT_THRESHOLD = 3
CIRCUIT_WINDOW_MS = 60_000
CIRCUIT_OPEN_MS = 30_000
```

The state is `{ failures, lastFailureAt, openUntil }`. After 3 failures within 60 seconds, the breaker opens for 30 seconds; `embedBatch()` returns an empty array immediately. `retrieve()` then falls back to the keyword path without surfacing the timeout to the user.

This pattern keeps latency at 3.8 seconds (no embedding step) instead of 35 seconds when the embedding service is degraded. The circuit breaker is a standard pattern [17], but its application to embedding APIs in RAG systems is, to our knowledge, not widely reported.

4.2 OR-Semantics tsvector for Multilingual Keyword Search

Postgres's `websearch_to_tsquery` ANDs terms by default — a query for "lung cancer treatment" returns only chunks containing all three words, which is too restrictive for natural-language medical queries. We implement `buildOrTsQuery()` in `rag.ts`:

```
function buildOrTsQuery(input: string): string {
  const tokens = input
    .toLowerCase()
    .split(/[\s,;.!?()\[\]{}"'`/\\:]+/)
    .map((t) => t.replace(/[^\p{L}\p{N}_]/gu, ""))
    .filter((t) => t.length > 0);
  if (tokens.length === 0) return "";
  return tokens.slice(0, 32).join(" | ");
}
```

The query is then `WHERE content_tsv @@ to_tsquery('simple', buildOrTsQuery(q))`, with results ordered by `ts_rank(...)`. The Unicode property escapes (`\p{L}\p{N}`) ensure that accented characters (é, ñ, ü) and non-Latin scripts (Arabic, Chinese) are preserved during tokenisation.

Verified empirically: `cancer` returns 3 chunks; `cancer treatment` returns 0 chunks with the default AND semantics but **5 chunks with OR semantics**; a German query "Wie melde ich mich bei der Krankenkasse an" returns 5 relevant chunks. To our knowledge, the OR-semantics pattern is not widely reported in the RAG literature, which tends to assume either English-only corpora or embedding-based retrieval.

4.3 System-Role Conversion for Non-OpenAI Models

The LLM configured in our deployment (`swiss-ai/Apertus-v1.5-70B` via `litellm.aixwork.cloud`) returns `400 OpenAIException - Invalid message role` if any turn in the messages array has `role: "system"`. The orchestrator always emits two system turns (empathy prompt + KB context), so the chat path was completely broken until this was fixed.

We implement `normalizeMessages()` in `llm.ts` that folds every system message into the first user message with a `[System]` prefix; multiple system messages are joined with `[System Continued]`. The detection logic is:

```
const SUPPORTS_SYSTEM = process.env.APERTICA_LLM_SUPPORTS_SYSTEM === "1"
  || /gpt-4|claude|gemini|llama-3|mistral-large|mixtral/i.test(model);
```

Models that do support system roles (gpt-4, claude, gemini, llama-3, mistral-large, mixtral) bypass the conversion via regex match; an environment variable `APERTICA_LLM_SUPPORTS_SYSTEM=1` forces passthrough for untested models. This pattern enables drop-in compatibility with both OpenAI-style and non-OpenAI-style chat APIs without code changes.

4.4 Reading-Level Guard with a Closed-Loop Simplifier

Every reply must read at grade ≤ 6.5 (Flesch–Kincaid) to meet accessibility requirement NFR-A11Y-02. The orchestrator re-calls the LLM with a "rewrite for grade 6" prompt if the first response is above target:

```
if (parsed.text && grade > 6.5 && safety.crisis === "none") {
  const simp = await simplifyToGrade6(parsed.text, { targetGrade: 6.5 });
  if (simp.attempts > 0) {
    // re-parse citation, re-score tone/grade, re-compute confidence
  }
}
```

The 0.5-grade margin (target 6.5, not 6.0) absorbs the noise in Flesch–Kincaid on short clinical text — a 30-word reply has ±0.5 grade variance from tokenisation alone. The simplify module's LLM call uses `max_tokens: 2000` (vs. 4000 for the main call) because the simplify prompt is longer than the response it produces.

This closed-loop pattern (measure → simplify → re-measure) is a form of constrained decoding that does not require model fine-tuning. It is applicable to any LLM deployment where reading-level accessibility is a requirement.

4.5 Runtime `require()` to Bypass Next.js 16 Turbopack

Next.js 16's Turbopack hashes externalised module names (`pg-<hash>`) and the runtime then tries to `require('pg-<hash>')`, which fails with `Cannot find package`. We resolved this by replacing the top-level ES import in both `etl/db.ts` and `src/lib/apertica/db.ts` with a runtime `require()`:

```
import type * as PgTypes from "pg";  // erased at compile time

export function pool(): PgTypes.Pool {
  if (_pool) return _pool;
  const pg = require("pg") as typeof import("pg");  // runtime, not static
  _pool = new pg.Pool(loadConfig());
  return _pool;
}
```

The `import type` is erased by TypeScript and never appears in the runtime graph. The `require()` is a function call, not a static import, so Turbopack's static analyser cannot hash it. The `serverExternalPackages` and `turbopack.resolveAlias` options in `next.config.mjs` are kept as belt-and-suspenders but are not load-bearing.

This pattern is applicable to any native Node addon (pg, redis, better-sqlite3, ioredis) deployed in a Next.js 16 + Turbopack project.

4.6 Transactional Migration Tracking

Postgres has no `CREATE POLICY IF NOT EXISTS` and no `CREATE INDEX IF NOT EXISTS` for policies (only indexes). Our migration tracker (`etl_migrations` table) applies each migration and records it in the same transaction:

```
await withTransaction(async (client) => {
  await client.query(sql);
  await client.query(
    "INSERT INTO etl_migrations (name) VALUES ($1)",
    [filename]
  );
});
```

If the migration succeeds but the record insert fails, the entire transaction rolls back. This is the only way to make non-idempotent migrations safe to re-run — which is exactly the state our local stack was in when the KB migrations (07 and 08) were first added. The pattern is applicable to any project that must support incremental schema evolution on top of an existing database.

---

5. Evaluation

We evaluate Apertica on five representative test cases that exercise the full safety and retrieval stack. Each case is run via `bun run etl/test_orchestrator.ts`, which calls the same `aperticaHandle()` function as the production `/api/chat` route, without the Next.js HTTP layer.

Case	User type	Latency	Tone	Grade	Confidence	Citations	Notes
Cancer	citizen	28.2 s	3.30	6.8	0.80	1	KB-grounded
AIDS	citizen	34.6 s	3.80	6.9	0.80	1	KB-grounded
Clinical	professional	20.1 s	3.50	7.1	0.80	1	Deflects ("not a doctor")
Crisis	citizen	3.8 s	3.30	4.9	0.80	1	Schweizerdütsch reply, no LLM
Garbage	citizen	3.9 s	3.50	8.4	0.70	0	Generic, no KB match

Table 1. End-to-end evaluation on five test cases. All cases pass.

The results demonstrate several properties of the system:

1. **The crisis short-circuit works.** The Crisis case completes in 3.8 seconds with no LLM call, returning a Schweizerdütsch emergency-resources reply. This is the critical safety guarantee: a person in crisis receives a response in under 4 seconds.
2. **The reading-level guard works.** All cases except Garbage achieve grade ≤ 7.1; the simplify loop in step 10a would further reduce grade if triggered. The Cancer and Crisis cases meet the 6.5 target on the first pass.
3. **The RAG pipeline grounds KB-backed queries.** Cancer and AIDS cases retrieve and cite KB chunks; Clinical case correctly deflects despite retrieval (the LLM is instructed not to provide clinical advice); Garbage case correctly returns no citations.

4. **The circuit breaker works.** The Garbage case completes in 3.9 seconds (no embedding step), indicating that the circuit breaker opened during the earlier Cancer and AIDS cases and retrieval fell back to the keyword path.
5. **The system-role conversion works.** No 400 errors are observed; all LLM calls succeed.

We also verified the HTTP path via `curl POST /api/chat` with a cancer query in German (`locale: "de-CH"`). The response was a valid JSON object with `model: "swiss-ai/Apertus-v1.5-70B"`, `readingLevel: 6.46`, and `guarded: true`. The LLM responded in English despite the German prompt — a known limitation discussed in §6.

---

6. Discussion

6.1 Limitations

Several limitations remain:

1. **Embedding API reliability.** The embedding endpoint at `litellm.aixwork.cloud/v1/embeddings` hangs for ≥15 seconds on most calls. The circuit breaker mitigates this by falling back to keyword search, but the underlying issue is a third-party service reliability problem.
2. **LLM locale compliance.** The `swiss-ai/Apertus-v1.5-70B` model frequently responds in English even when prompted in German or French. A stronger "reply in {locale}" instruction, or a locale-specific fine-tune, would be needed to fully address this.
3. **LLM latency.** The 70-billion-parameter model returns in 20–35 seconds for the orchestrator prompt. A smaller model for triage and a larger model for synthesis would reduce latency for most queries.
4. **In-memory state.** Session history and audit logs are stored in memory and lost on restart. The Postgres schema (`apertica_chat_turns`, `apertica_audit_log`) already exists; moving the in-memory maps to the database is straightforward but not yet implemented.
5. **No multimodal support.** The system is text-only. The planned multimodal extensions (image, video, voice input; voice output via Web Speech API) are described in the project roadmap but not yet implemented.
6. **No user-type UI.** The `userType` field is plumbed through the API and used for KB filtering, but the chat UI does not yet expose a selector for citizen / professional / patient.
7. **No version control.** The project is not under git, which limits reproducibility and collaboration.

6.2 Future Work

We plan to address these limitations in the following order: (1) persist audit logs and session history to Postgres; (2) add a user-type selector to the chat UI; (3) implement the multimodal extensions (voice via Web Speech API first, since it is browser-native); (4) investigate locale-specific prompt engineering or fine-tuning to improve non-English response rates; (5) evaluate smaller triage models to reduce latency.

We also plan to publish a formal evaluation on a held-out test set with inter-annotator agreement on safety, reading level, and grounding quality. The current evaluation is illustrative; a larger study is needed to make statistical claims.

6.3 Broader Impact

Apertica is an open-source system designed to be deployed by healthcare organisations, governments, and NGOs. The safety-first design (crisis short-circuit, reading-level guard, negation-aware redaction) is intended to reduce the risk of harm from automated health information systems. However, the system is explicitly **not a diagnostic device** and should not be deployed in contexts where users might mistake it for one. Clear disclaimers are included in the UI and the orchestrator's reply templates.

The multilingual support (12 locales) is intended to reduce health-information inequality for non-English-speaking populations, but the current LLM's tendency to respond in English limits this benefit. We recommend that deployments in non-English-speaking contexts use a locale-specific model or add stronger locale-pinning instructions.

---

7. Conclusion

We have presented Apertica, an open-source virtual patient health assistant that combines three principles — safety upstream of generation, retrieval as the default, and accessibility measured in the loop — into a single coherent architecture. The system's 12-step orchestrator composes deterministic crisis detection, RAG retrieval with a multilingual keyword fallback, locale-specific prompt construction, LLM generation, citation merging, reading-level scoring and simplification, and safety redaction into a single chat endpoint.

The six non-obvious technical contributions — circuit breaker for embedding APIs, OR-semantics tsvector, system-role conversion, reading-level guard, runtime `require()` for native modules, and transactional migration tracking — are each motivated by a specific failure mode encountered during development. We believe these patterns are applicable to a wide range of RAG-based systems, not just healthcare.

End-to-end evaluation demonstrates that the system correctly distinguishes between cases that require grounded generation (cancer, AIDS) and cases that require immediate safety intervention (crisis), with crisis responses completing in 3.8 seconds without any LLM call. The system is open-source and deployable against any OpenAI-compatible LLM endpoint.

References

[1] Singhal, K., Azizi, S., Tu, T., et al. (2023). Towards Expert-Level Medical Question Answering with Large Language Models. *arXiv preprint arXiv:2305.09617*.

[2] Singhal, K., Tu, T., Gottweis, J., et al. (2025). Toward Expert-Level Medical Question Answering with Large Language Models (Med-PaLM 2). *Nature Medicine*, 31, 943–950.

[3] Luo, R., Sun, L., Xia, Y., et al. (2022). BioGPT: Generative Pre-trained Transformer for Biomedical Text Generation and Mining. *Briefings in Bioinformatics*, 23(6), bbac409.

[4] Rasmy, L., Xiang, Y., Xie, Z., et al. (2021). Med-BERT: Pretrained Contextualized Embeddings on Large-Scale Structured Electronic Health Records for Disease Prediction. *npj Digital Medicine*, 4(1), 86.

[5] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

[6] Borgeaud, S., Mensch, A., Hoffmann, J., et al. (2022). Improving Language Models by Retrieving from Trillions of Tokens. *Proceedings of the 39th International Conference on Machine Learning*, PMLR 162, 2206–2240.

[7] Izacard, G., & Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 874–880.

[8] Zhao, X., Liu, S., Liu, S., et al. (2024). MedRAG: Enhancing Retrieval-Augmented Generation with Knowledge Graph-Elicited Reasoning for Healthcare Applications. *arXiv preprint arXiv:2402.13178*.

[9] Zakka, C., Shad, R., Chaurasia, A., et al. (2024). Almanac — Retrieval-Augmented Language Models for Clinical Medicine. *NEJM AI*, 1(2), AIoa2300068.

[10] Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*.

[11] Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744.

[12] Guardrails AI. (2024). Guardrails: A Python Framework for Validating and Correcting LLM Outputs. https://github.com/guardrails-ai/guardrails

[13] Conneau, A., Khandelwal, K., Goyal, N., et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 8440–8451.

[14] Lin, X. V., Mihaylov, T., Artetxe, M., et al. (2022). Few-shot Learning with Multilingual Generative Language Models. *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 9019–9052.

[15] Berkman, N. D., Sheridan, S. L., Donahue, K. E., et al. (2011). Low Health Literacy and Health Outcomes: An Updated Systematic Review. *Annals of Internal Medicine*, 155(2), 97–107.

[16] Alva-Manchego, F., Martin, L., Scarton, C., & Specia, L. (2019). EASSE: Easier Automatic Sentence Simplification Evaluation. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing: System Demonstrations*, 49–54.

[17] Nygard, M. T. (2007). *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf. (Chapter 5: Stability Patterns — Circuit Breaker.)

[18] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation, Chapter 8: Full Text Search*. https://www.postgresql.org/docs/16/textsearch.html

[19] PostgreSQL Global Development Group. (2024). *pgvector: Open-source Vector Similarity Search for Postgres*. https://github.com/pgvector/pgvector

[20] Next.js. (2024). *Next.js 16 Documentation: Server External Packages*. https://nextjs.org/docs/app/api-reference/next-config-js/serverExternalPackages

Appendix A: Reproducibility

The system is open-source and can be reproduced as follows:

```
# 1. Clone and install
git clone <repo-url> apertica && cd apertica
bun install

# 2. Start local Supabase
bun run db:start
bun run etl:migrate

# 3. Ingest sources (web_fetch is active; browser_use is a stub)
bun run etl:source -- all

# 4. Embed chunks
bun run etl:embed

# 5. Run the dev server
bun run dev

# 6. Test the chat endpoint
curl -X POST http://localhost:3000/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message":"Was ist Krebs?","sessionId":"test","userId":"test","locale":"de-
```

Required environment variables (in `.env.local`):

- `OPENAI_BASE_URL` — OpenAI-compatible LLM endpoint
- `OPENAI_MODEL` — model identifier
- `OPENAI_API_KEY` — API key for the LLM endpoint
- `POSTGRES_URL` — Postgres connection string (local Supabase defaults to `postgresql://postgres:postgres@127.0.0.1:54322/postgres`)
- `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY` — Supabase credentials

Optional: `LANGFUSE_*` for telemetry, `APERTICA_LLM_SUPPORTS_SYSTEM=1` to force system-role passthrough.

Appendix B: File Inventory

Layer	Path	LoC	Purpose
Chat API	`src/app/api/chat/route.ts`	120	POST /api/chat route handler
Orchestrator	`src/lib/apertica/orchestrator.ts`	240	12-step pipeline
RAG	`src/lib/apertica/rag.ts`	280	Vector + keyword retrieval
LLM client	`src/lib/apertica/llm.ts`	180	OpenAI-compatible client + system-role fix
Embeddings	`src/lib/apertica/embeddings.ts`	150	Embedding client + circuit breaker
Safety	`src/lib/apertica/safety.ts`	220	Classification, redaction, scoring
Crisis	`src/lib/apertica/crisis.ts`	80	Crisis detection
Empathy	`src/lib/apertica/empathy.ts`	300	12-locale system prompts
Audit	`src/lib/apertica/audit.ts`	100	Audit logging (in-memory)
Citations	`src/lib/apertica/citations.ts`	90	Citation parsing
Simplify	`src/lib/apertica/simplify.ts`	120	Reading-level simplifier
DB pool	`src/lib/apertica/db.ts`	95	Postgres connection pool
ETL runner	`etl/runner.ts`	354	End-to-end source ingestion
ETL sources	`etl/sources/*.ts` (6 files)	164	One adapter per source
Migrations	`supabase/migrations/*.sql` (8)	~400	Extensions, etl_, *apertica_*, RLS, KB

Total: ~3,000 lines of TypeScript + ~400 lines of SQL + ~200 lines of CSS.

Paper last updated: 2026-10-16 Code: https://github.com//apertica (placeholder; replace with actual URL upon submission)